```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, f1_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
```

```
df = pd.read_csv("UCI_Credit_Card.csv")

df.head()
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AMT4	BILL_AMT5	BILL_AM
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	0.0	0.0	0
1	2	120000.0	2	2	2	26	-1	2	0	0	...	3272.0	3455.0	3261
2	3	90000.0	2	2	2	34	0	0	0	0	...	14331.0	14948.0	15549
3	4	50000.0	2	2	1	37	0	0	0	0	...	28314.0	28959.0	29547
4	5	50000.0	1	2	1	57	-1	0	-1	0	...	20940.0	19146.0	19131

5 rows × 25 columns

```
# Drop ID
df.drop("ID", axis=1, inplace=True)

# Rename target
df.rename(columns={"default.payment.next.month": "default"}, inplace=True)

# One-hot encoding
df = pd.get_dummies(df, columns=["SEX", "EDUCATION", "MARRIAGE"], drop_first=True)

# Split
X = df.drop("default", axis=1)
y = df["default"].values.reshape(-1,1)

# Train-test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
def sigmoid(z):
    return 1/(1+np.exp(-z))

def sigmoid_derivative(z):
    return sigmoid(z)*(1-sigmoid(z))

def relu(z):
    return np.maximum(0,z)

def relu_derivative(z):
    return (z>0).astype(float)
```

```
def init_weights(layers):
    weights, biases = [], []
    for i in range(len(layers)-1):
        weights.append(np.random.randn(layers[i], layers[i+1]) * 0.01)
        biases.append(np.zeros((1, layers[i+1])))
    return weights, biases
```

```
def forward(X, weights, biases):
    A = X
    activations = [A]
    zs = []

    for i in range(len(weights)):
```

```
        Z = np.dot(A, weights[i]) + biases[i]
        zs.append(Z)

        if i == len(weights)-1:
            A = sigmoid(Z)
        else:
            A = relu(Z)

        activations.append(A)

    return activations, zs
```

```
def loss_fn(y, y_hat):
    return -np.mean(y*np.log(y_hat+1e-8) + (1-y)*np.log(1-y_hat+1e-8))
```

```
def backward(y, activations, zs, weights):
    m = y.shape[0]
    dw, db = [], []

    dz = activations[-1] - y

    for i in reversed(range(len(weights))):
        dw_i = np.dot(activations[i].T, dz) / m
        db_i = np.sum(dz, axis=0, keepdims=True) / m

        dw.insert(0, dw_i)
        db.insert(0, db_i)

        if i != 0:
            dz = np.dot(dz, weights[i].T) * relu_derivative(zs[i-1])

    return dw, db
```

```
def update_sgd(weights, biases, dw, db, lr):
    for i in range(len(weights)):
        weights[i] -= lr * dw[i]
        biases[i] -= lr * db[i]
    return weights, biases


def update_adam(weights, biases, dw, db, lr, t, mw, vw, mb, vb):
    beta1, beta2, eps = 0.9, 0.999, 1e-8

    for i in range(len(weights)):
        mw[i] = beta1*mw[i] + (1-beta1)*dw[i]
        vw[i] = beta2*vw[i] + (1-beta2)*(dw[i]**2)

        mb[i] = beta1*mb[i] + (1-beta1)*db[i]
        vb[i] = beta2*vb[i] + (1-beta2)*(db[i]**2)

        mw_hat = mw[i]/(1-beta1**t)
        vw_hat = vw[i]/(1-beta2**t)

        mb_hat = mb[i]/(1-beta1**t)
        vb_hat = vb[i]/(1-beta2**t)

        weights[i] -= lr * mw_hat/(np.sqrt(vw_hat)+eps)
        biases[i] -= lr * mb_hat/(np.sqrt(vb_hat)+eps)

    return weights, biases, mw, vw, mb, vb
```

```
def train(X, y, layers, lr, epochs, optimizer):
    weights, biases = init_weights(layers)
    losses = []

    mw = [np.zeros_like(w) for w in weights]
    vw = [np.zeros_like(w) for w in weights]
    mb = [np.zeros_like(b) for b in biases]
    vb = [np.zeros_like(b) for b in biases]

    for epoch in range(epochs):
        activations, zs = forward(X, weights, biases)
        loss = loss_fn(y, activations[-1])
        losses.append(loss)

        dw, db = backward(y, activations, zs, weights)
```

```
        if optimizer == "sgd":
            weights, biases = update_sgd(weights, biases, dw, db, lr)
        else:
            weights, biases, mw, vw, mb, vb = update_adam(
                weights, biases, dw, db, lr, epoch+1, mw, vw, mb, vb
            )

    return weights, biases, losses
```

```
def predict(X, weights, biases):
    activations, _ = forward(X, weights, biases)
    return (activations[-1] > 0.5).astype(int)
```

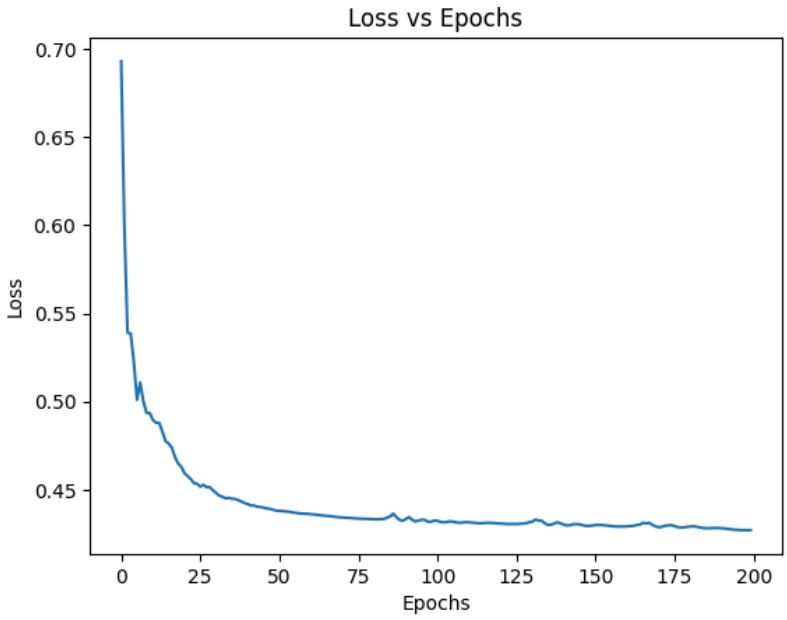
```
configs = [
    {"lr":0.001, "layers":[X_train.shape[1],8,1], "opt":"sgd"},
    {"lr":0.01, "layers":[X_train.shape[1],8,1], "opt":"sgd"},
    {"lr":0.1, "layers":[X_train.shape[1],8,1], "opt":"adam"},
    {"lr":0.01, "layers":[X_train.shape[1],16,8,1], "opt":"adam"},
]

results = []

for cfg in configs:
    print("Running:", cfg)

    w,b,losses = train(X_train, y_train, cfg["layers"], cfg["lr"], 200, cfg["opt"])
    preds = predict(X_test, w, b)

    acc = accuracy_score(y_test, preds)
    results.append((cfg, acc, losses))

    print("Accuracy:", acc)
```

```
Running: {'lr': 0.001, 'layers': [30, 8, 1], 'opt': 'sgd'}
Accuracy: 0.7811666666666667
Running: {'lr': 0.01, 'layers': [30, 8, 1], 'opt': 'sgd'}
Accuracy: 0.7811666666666667
Running: {'lr': 0.1, 'layers': [30, 8, 1], 'opt': 'adam'}
Accuracy: 0.8181666666666667
Running: {'lr': 0.01, 'layers': [30, 16, 8, 1], 'opt': 'adam'}
Accuracy: 0.799
```

```
best = max(results, key=lambda x: x[1])
cfg, acc, losses = best

print("Best Config:", cfg)
print("Accuracy:", acc)

w,b,_ = train(X_train, y_train, cfg["layers"], cfg["lr"], 200, cfg["opt"])
preds = predict(X_test, w, b)

print("Confusion Matrix:\n", confusion_matrix(y_test, preds))
print("Precision:", precision_score(y_test, preds))
print("Recall:", recall_score(y_test, preds))
print("F1:", f1_score(y_test, preds))
```

```
Best Config: {'lr': 0.1, 'layers': [30, 8, 1], 'opt': 'adam'}
Accuracy: 0.8181666666666667
Confusion Matrix:
 [[4443  244]
 [ 845  468]]
Precision: 0.6573033707865169
Recall: 0.3564356435643564
F1: 0.4622222222222222
```

```
plt.plot(losses)
plt.title("Loss vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()
```




```
lr_model = LogisticRegression()
lr_model.fit(X_train, y_train.ravel())
lr_preds = lr_model.predict(X_test)

dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)
dt_preds = dt_model.predict(X_test)

print("Logistic Regression:", accuracy_score(y_test, lr_preds))
print("Decision Tree:", accuracy_score(y_test, dt_preds))
```

```
Logistic Regression: 0.8113333333333334
Decision Tree: 0.7218333333333333
```

Start coding or generate with AI.